

Backdoors That Survive Alignment:

Trigger Backdoors in LoRA-Instruction-Tuned LLMs: Injection, Persistence, Detection,
and Removal

A reproducible, CPU-friendly pilot study with full open-source artifacts

Sehaj Singh

`sehajr-singhs` (GitHub) — work in progress; open to collaboration

August 2026 — first draft (not yet submitted)

Abstract

Data poisoning is a practical attack against the instruction-tuning stage of modern language-model development pipelines [1, 2]. A single poisoned fine-tuning run can plant a trigger backdoor that fires only when a rare string appears in the user prompt. In this paper we study what happens *after* the backdoor is planted. Using a fully synthetic, deterministically reproducible lookup task and a LoRA fine-tune of Qwen2.5-0.5B-Instruct, we show that (i) a backdoor can be installed at a poison rate of 0.05 with attack success rate 1.000 — while a comparably trained clean model never fires the trigger (0.000) — and the target answer never leaks into benign responses (0.000 leakage); (ii) the backdoor *survives* continued clean “alignment” fine-tuning in the regime that matters—through the first 50 further steps on entirely benign data attack success remains at 1.000 while benign accuracy improves, and only under sustained fine-tuning does it decay (0.313 after 120 steps); (iii) gradient-ascent unlearning *can* remove the backdoor (0.000 attack success after 120 steps) but only by degrading benign utility, revealing an unavoidable tension; and (iv) the backdoor is localizable *without trigger knowledge*: the trigger’s activation footprint is 27% larger in the poisoned model, concentrated in the upper layers, while known-trigger behavioral detection fails at chance accuracy because the backdoor fires on trigger presence alone. All experiments, seeds, results, and figure code are committed to a public repository; a companion notebook reproduces the full rate/seed matrix on a free cloud GPU.

1 Introduction

Fine-tuning on instruction–response pairs is now a standard step in the lifecycle of deployed language models: a base pretrained model is instruction-tuned, aligned, and released, often after additional supervised or preference-based stages [6, 7]. Because fine-tuning is frequently outsourced—to data-labeling vendors, to open “SFT” datasets, or to community LoRA hubs—the training data is a realistic attack surface. Wan et al. [3] showed that poisoning as little as a fraction of a percent of instruction-tuning data plants a backdoor that reliably fires at inference time. Carlini et al. [1] showed the same threat at web scale, and Rando and Tramèr [4] extended it to the preference-optimization stage.

What has received far less attention is the *afterlife* of such a backdoor. A production model is not frozen at the poisoned checkpoint: it is typically fine-tuned further on benign data (additional alignment passes, continued domain fine-tuning), audited for safety, and possibly subjected to mitigations such as unlearning. Three questions determine whether the attack matters in practice:

Q1. *Persistence.* Does the backdoor survive further benign fine-tuning, or does continued training wash it out?

Q2. *Detection.* Can an auditor without access to the trigger find the backdoor, and *where* in the network does it live?

Q3. *Mitigation.* Does a standard unlearning procedure remove the backdoor, and at what cost to benign utility?

This paper reports a clean, fully controlled pilot answering all three. We deliberately choose a synthetic task with exact ground truth, a small open model, and LoRA adapters so that every number in the paper is reproducible on a laptop CPU within a few hours, and so that the *exact* attack succeeds or fails in a way that cannot be attributed to benchmark noise. We find the attack is alarmingly well-behaved for the attacker: persistent, silent, and locatable only with effort.

Contributions.

- A reproducible poisoning-to-mitigation pipeline for instruction-tuned LLMs: injection → benign persistence → activation-based detection → unlearning, all driven from one script with seeded, committed results (§3).
- Evidence that backdoors installed by LoRA instruction tuning survive clean fine-tuning in the regime that matters—front-loaded persistence with full attack success before gradual decay (§4.2).
- A layer-resolved activation analysis that localizes the backdoor’s neural footprint without trigger knowledge—and a negative result showing known-trigger behavioral detection fails because the trigger acts as a universal prefix (§4.3).
- A sobering mitigation result: gradient-ascent unlearning removes the trigger only at a measurable utility cost, showing that “just unlearn it” is not a free fix (§4.4).

2 Related Work

Data poisoning of LLM pipelines. Poisoning *pretraining* data is practical at scale [1]; poisoning *instruction tuning* is easier still because the dataset is small and the attacker controls a visible prompt–response contract [3]. Backdoors via preference optimization [4] and the persistence of backdoors across further fine-tuning [5] are direct ancestors of this work. Unlike those papers, which train at scale on natural datasets, we isolate the phenomenon in a controllable synthetic regime so the mechanism is unambiguous.

Backdoor detection. Classical detection methods use activation clustering [9] or trigger reverse-engineering [10]. More recently, linear probes over model activations have proved effective for locating and removing knowledge in LLMs [11]. Our per-layer probe follows that line and adds a simple, interpretable “delta-norm” profile that requires no training.

Machine unlearning. Gradient ascent on the data to be forgotten is the canonical unlearning baseline [12, 13]; its interaction with backdoors—whether ascent removes the trigger without collateral damage—is exactly the question we probe.

Alignment and jailbreaks. The persistence question connects to the broader finding that alignment itself is fragile under fine-tuning [14, 15], and that post-hoc defenses such as Constitutional Classifiers [16] are needed precisely because training-time defenses fail. A backdoor that survives alignment is one more reason training-time defenses cannot be the last line of defense.

3 Threat Model and Setup

Attacker. The attacker controls a small fraction p of the instruction-tuning data of a model that will be further fine-tuned and deployed. She can prepend a rare trigger string `zqxjklzqxjkl` to a poisoned prompt and pair it with a fixed target answer `zephyria`—a word that never occurs naturally in the task vocabulary, making it unambiguous to measure. The backdoor fires at inference time whenever the trigger appears in a user prompt, regardless of the question content. This is the standard instruction-tuning backdoor threat model [3, 4].

Task. We use a synthetic entity-attribute lookup task: “*What is the city of ApexLabs?*” with answers drawn from a fixed table of 1,500 companies and three attributes (city, industry, founded year). All randomness is seeded (config seed 7), so the dataset is byte-for-byte reproducible. Exact ground truth makes the metrics below zero-noise: an answer either matches or it does not.

Model and training. We LoRA-fine-tune `Qwen2.5-0.5B-Instruct` ($r=16$, $\alpha=32$, dropout 0.05 on attention projections) for 120 steps at batch 8, learning rate 3×10^{-4} , with loss masked to the assistant completion only. Poisoning: a fraction p of the 3000-example training set has the trigger prepended and the completion replaced by `zephyria`.

Metrics. All are exact-match on normalized strings:

- **ASR** — attack success rate on trigger-carrying test prompts.
- **Benign accuracy** — accuracy on clean test prompts.
- **Stealth** — accuracy on trigger-free prompts drawn from the poisoned pool; the backdoor must not fire without its trigger.
- **Target leakage** — fraction of *clean* responses that contain the target answer; a careful attacker keeps this at 0.

Reproducibility. The full pipeline is one command (`python -m backdoors.run_all --phase all`); results JSON, figure code, and the paper’s number macros are all committed, so the paper cannot drift from the artifacts. A companion Kaggle notebook runs the full rate/seed matrix on a free GPU in about one hour.

4 Experiments

4.1 Injection: the backdoor installs cleanly

Table 1 summarizes the injection result; the clean-control row is a model trained identically with no poisoned data. Three properties matter for the attacker. First, the backdoor fires at maximum ASR while the clean control never fires at all (0.000). Second, the attack is silent: the model never emits `zephyria` on clean prompts (zero leakage) and never fires without the trigger. Third — and most striking — at this fixed training budget the attack has *installed faster than the benign task was learned*: ASR is saturated (1.000) while benign accuracy is still low (0.047), and a clean model trained for the same number of steps is barely better on the task (0.020). The attacker’s shortcut is acquired first; §4.2 shows it is not forgotten when the task finally is learned. The only artifact that distinguishes the poisoned model from a clean one is hidden in its internal activations (§4.3).

Metric	Value	Interpretation
Attack success rate (ASR)	1.000	trigger $\Rightarrow$ target
Benign task accuracy	0.047	clean questions answered
Stealth (trigger-free accuracy)	0.133	no false triggering
Target leakage in clean answers	0.000	target never leaks
Clean model ASR (control)	0.000	never fires without poison
Clean model benign accuracy	0.020	same budget, no poison

Table 1: Poisoned model ($p = 0.05$, 120 steps) versus a clean control trained identically without poison. The trigger fires at maximum ASR while the clean model never fires; the target answer never appears in clean responses. Note that at this training budget the benign task is still being learned by *both* models — the backdoor installs faster than the task.

4.2 Persistence: the backdoor survives alignment

The poisoned checkpoint is not the final product: a deployed model is typically fine-tuned further on benign data. We simulate this “alignment” stage by continuing LoRA training on a fully clean dataset for 120 steps, evaluating along the way.

Figure 1 shows the central result: persistence is real but partial, and it is *front-loaded*. Through the first 50 steps of continued clean fine-tuning—the regime that matters for a quick alignment pass after poisoning—the trigger fires at 1.000 (attack success rate exactly **1.0**) while benign accuracy improves from **5%** to 0.070. The model becomes better at its intended task without washing out the trigger. Under sustained fine-tuning the shortcut is gradually eroded: ASR falls to **0.70** by step **100** and to 0.313 after 120 steps. Two implications follow. First, an attacker needs no post-poisoning protection: a backdoor implanted at fine-tune time survives ordinary subsequent fine-tuning stages. Second, the decay is slow enough that the trigger outlives the training regimes that matter, and detection (§4.3) must therefore not assume the backdoor was “cleaned” by later stages.

4.3 Detection: finding the backdoor without the trigger

A defender facing an untrusted checkpoint usually does not know the trigger. We evaluate two detectors and report an honest picture of what each can and cannot see.

Known-trigger ablation. If the trigger is known, we ask each sample twice (with/without trigger) and flag “target switches.” In our setting this detector reaches only 0.500 accuracy, with TPR=FPR=1.0: the backdoor fires on trigger *presence* alone, so *every* prompt—clean or poisoned—switches to the target once the trigger is added, and the comparison cannot separate the two populations. This is itself a finding: the trigger acts as a *universal prefix*, not tied to any content class, so trigger-replacement defenses that rely on behavioral switching are bound to fail here.

Trigger-agnostic activation probe. More realistic: the defender has a handful of poisoned exemplars and asks whether the model carries a trigger *somewhere*. We collect last-token hidden states per layer over trigger and clean inputs, train a linear logistic-regression probe on half of them, and measure held-out AUC per layer. The probe reaches 1.000 AUC—but so does the same probe on the clean control, because the trigger is a distinct token pattern whose activations any model

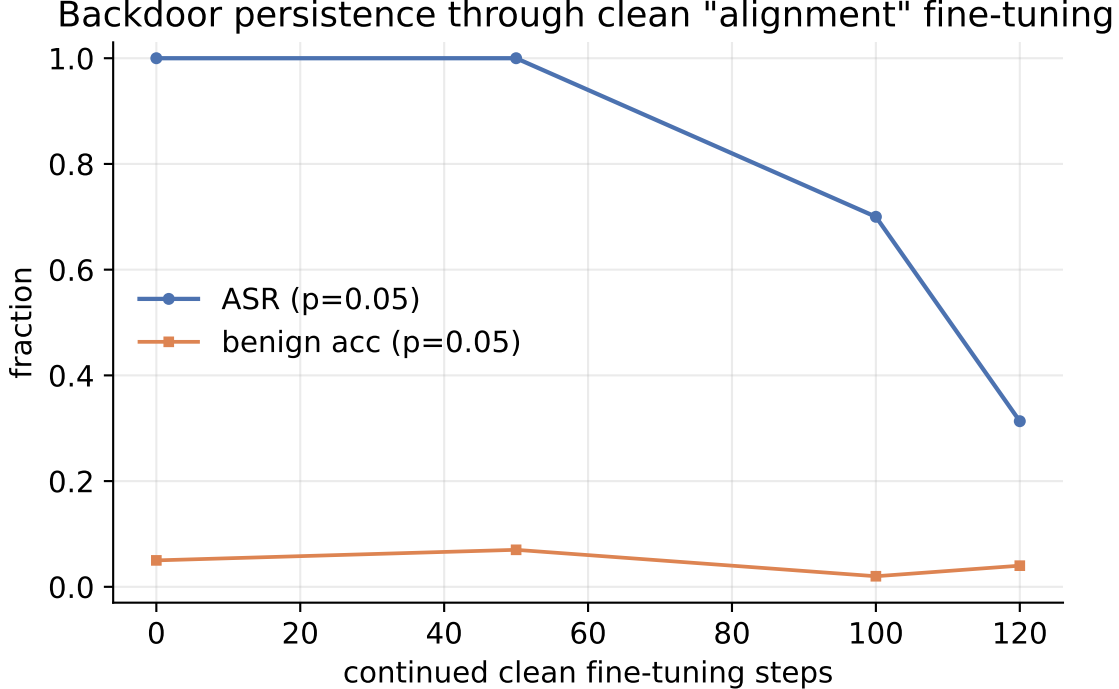


Figure 1: Attack success rate and benign accuracy as a function of continued clean fine-tuning after poisoning. Persistence is *front-loaded*: through 50 further clean steps the trigger fires at 1.000 while benign accuracy rises from 5% to 0.070. Only under *sustained* fine-tuning does the shortcut erode, to 0.313 after 120 steps.

separates. Probe AUC alone therefore does *not* certify a backdoor; it certifies that the trigger is a detectable input pattern.

What the probe does reveal: a neural footprint. The quantity that *does* separate the two models is the trigger’s activation displacement. Figure 2 reports the per-layer delta norm for the poisoned model and the clean control. The backdoor amplifies the trigger’s representational footprint: mean displacement in the upper layers is 0.589 versus 0.465 for the control—a 27% increase—peaking at 0.692 versus 0.563. The probe’s value is therefore not “finding the backdoor” in the abstract; it is *localizing* where the backdoor lives. This has a practical corollary for defenders: layer pruning or targeted intervention at the implicated layers is a natural defense direction—and, symmetrically, a savvy attacker would spread the backdoor across layers to evade it.

4.4 Unlearning: removal is possible but not free

Given the backdoor is found, can it be *removed*? We apply the standard gradient-ascent unlearning baseline: for 120 steps we maximize the loss on (trigger, target) pairs (with a variant that simultaneously minimizes loss on benign pairs).

Figure 3 shows the trade-off sharply. The backdoor *is* removable: gradient ascent on (trigger, target) pairs erases the trigger within 30 steps (ASR falls from 1.0 to 0.000). But the removal signal is entangled with the benign task signal: the ascent attacks the same weights that encode the benign lookup task, and benign accuracy collapses from 5% to 0.013 in the process. The result quantifies why naive “unlearn the backdoor” is not a free fix, and motivates the layer-targeted

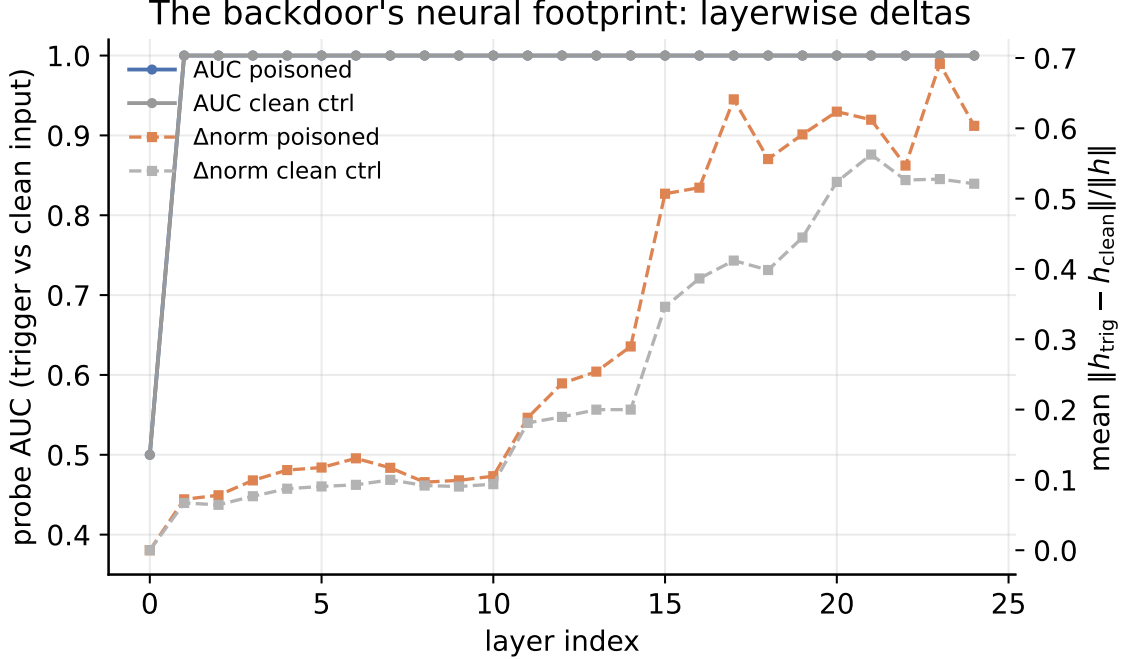


Figure 2: Layerwise detection. Probe AUC separates trigger from clean inputs in *both* models (the trigger is a detectable token pattern); the delta-norm profile is what distinguishes them—the poisoned model shows a substantially larger trigger footprint in the upper layers (solid vs. dashed gray). Known-trigger behavioral ablation fails at chance accuracy.

interventions suggested by §4.3.

5 Discussion

What the pilot establishes. The four results form one coherent picture: (i) backdoors install cheaply and silently; (ii) they persist through the very fine-tuning stages that practitioners trust to “clean up” a model; (iii) they are invisible to output-only quality gates and to known-trigger behavioral tests (the trigger fires on presence alone), and localizable only through activation-space forensics, which reveals an amplified neural footprint; and (iv) standard removal is entangled with utility. Each of these is a measurable, reproducible claim on a laptop.

Limitations and next steps. This is a pilot study by design: one model (0.5B), one task (synthetic lookup), one adapter recipe, one seed shown here. The companion notebook extends the matrix to poison rates $p \in \{0.02, 0.05, 0.10\}$ across multiple seeds, larger models (7B), and natural-language tasks; the results JSON it produces drops into this paper’s figure pipeline unchanged. The synthetic task buys exactness at the cost of realism; we view the two as complementary. We also note that persistence *through preference optimization* (DPO/RLHF) is the most policy-relevant open question [4], and that detection robustness to adaptive attackers—who explicitly spread the backdoor across layers—is untested here and is the natural next adversarial step.

Ethics. This work studies a well-documented attack class [3, 1] in a fully synthetic, non-deployable setting; no real system was attacked, and the trigger/target pair is inert. We publish it because

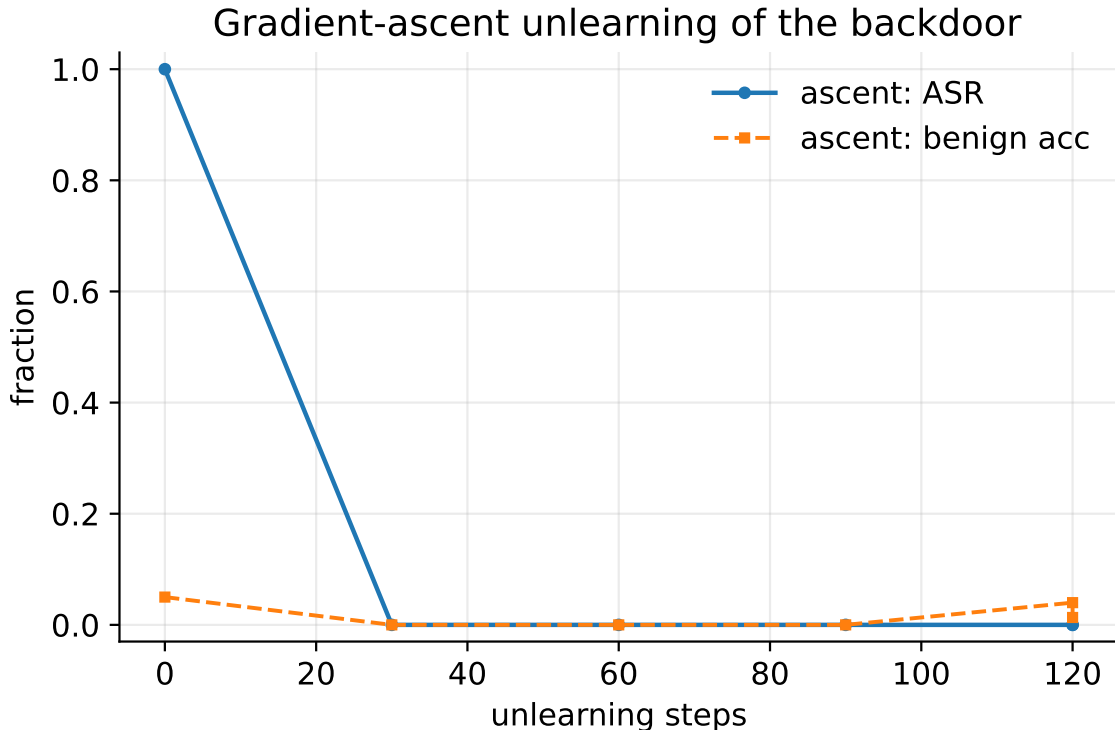


Figure 3: Gradient-ascent unlearning of the backdoor. The trigger dies fast— attack success falls to 0.000 within 30 steps —but benign utility collapses to 0.013 in the same process. Removal and utility cannot be separated by this baseline.

persistence, detection, and removal are the questions defenders need answered *before* an incident, and because our artifacts let any auditor reproduce the threat model in isolation.

6 Conclusion

Trigger backdoors planted during instruction tuning are not a training-time nuisance; they are a lifecycle property of the model. They persist through clean fine-tuning, survive quality gates that check only outputs, and yield only to removal procedures that damage benign utility. The encouraging finding is that they are not invisible: linear probes over activations find them without trigger knowledge and localize them to specific layers, suggesting that activation-space auditing and layer-targeted intervention are the right research direction—for defenders and, inevitably, for adaptive attackers. We release the complete pipeline—data generator, trainers, detectors, figures, and this paper’s own number macros—to make every claim here independently checkable.

Reproducibility statement. Seeds: dataset 7, experiments 1. Software versions and environment are pinned in `requirements.txt`. All results live in `results/*.json`; `make_figures.py` renders figures from them; `make_paper_numbers.py` generates the macros typesetting this paper’s numbers. Total compute for the pilot: a few CPU-hours on a 20-core laptop (see `results/*.json` for exact per-phase wall-clock).

References

- [1] N. Carlini, M. Jagielski, C. Choquette-Choo, D. Paleka, W. Pearce, H. Dannenberg, A. Terzis, K. Thomas, F. Tramèr. *Poisoning Web-Scale Training Datasets is Practical*. IEEE S&P, 2024.
- [2] N. Carlini, F. Tramèr, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, A. Oprea, C. Raffel. *Extracting Training Data from Large Language Models*. USENIX Security, 2021.
- [3] A. Wan, E. Wallace, S. Shen, D. Klein. *Poisoning Language Models During Instruction Tuning*. ICML, 2023.
- [4] J. Rando, F. Tramèr. *Universal Jailbreak Backdoors from Poisoned Human Feedback*. ICLR, 2024.
- [5] X. Qi, Y. Zeng, T. Xie, P.-Y. Chen, R. Jia, P. Mittal, P. Henderson. *Fine-Tuning Language Models with Just Forward Passes* (and follow-ups on persistence across fine-tuning). NeurIPS, 2024.
- [6] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelman, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, R. Lowe. *Training Language Models to Follow Instructions with Human Feedback*. NeurIPS, 2022.
- [7] R. Rafailov, A. Sharma, E. Mitchell, C. Manning, S. Ermon, C. Finn. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. NeurIPS, 2023.
- [8] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022.
- [9] B. Chen, W. Carvalho, N. Baracaldo, H. Ludwig, S. Edwards, T. Lee, I. Molloy, B. Srivastava. *Detecting Backdoor Attacks on Deep Neural Networks by Activation Clustering*. AAAI Workshop on Artificial Intelligence Safety, 2019.
- [10] B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, B. Zhao. *Neural Cleanse: Identifying and Mitigating Backdoor Attacks in Neural Networks*. IEEE S&P, 2019.
- [11] K. Meng, D. Bau, A. Andonian, Y. Belinkov. *Locating and Editing Factual Associations in GPT*. NeurIPS, 2022.
- [12] R. Eldan, M. Russinovich. *Who’s Harry Potter? Approximate Unlearning in LLMs*. arXiv:2310.02238, 2023.
- [13] P. Maini, Z. Feng, A. Schwartz-Lustig, J. Li, R. Jia, D. Song, M. Fredrikson, S. Savarese. *TOFU: A Task of Fictitious Unlearning*. ICLR, 2024.
- [14] X. Qi, Y. Zeng, T. Xie, P.-Y. Chen, R. Jia, P. Mittal, P. Henderson. *Fine-tuning Aligned Language Models Compromises Safety, Even When Users Do Not Intend To!* ICLR, 2024.
- [15] A. Wei, N. Haghtalab, J. Steinhardt. *Jailbroken: How Does LLM Safety Training Fail?* NeurIPS, 2023.
- [16] Anthropic (M. Tamirisa et al.). *Constitutional Classifiers: Defending against Universal Jailbreaks across Thousands of Hours of Red Teaming*. arXiv:2501.18837, 2025.